Centralized Configuration

Lets see the problems first:

<u>Problem1</u>: Rebuild and Redeploying

Order application

```
server.port=8081
spring.application.name=order-service

custom.message=hello world

```

```
@Service
public class OrderService {

    @Value("${custom.message}")
    private String message;

    public String getOrder() {
        return "fetch the Order details with message:" + message;
    }
}

```

Concept && Coding

If config are kept within microservice resource(application.properties), then any change means:

- Edit *application.property* file
- Rebuild the JAR
- Redeploy service

<u>Problem2</u>: Inconsistent config across services

Pre-Order application

```
db.url=jdbc:mysql://localhost:3306/order-db
```

Post-Order application

```
db.url=jdbc:mysql://localhost:3306/order-db
```

say in above use case both the service has common configuration "*db.url*" but when any update is needed both service *application.properties* need to be changed.

<u>Problem3</u>: No Runtime update

- Local *application.properties* is loaded **only once at startup**.
- If value changes (e.g. endpoint URL) app must restart.

<u>Problem4</u>: Time consuming rollback

- Each service keeps its own application.properties locally and suppose we change:
  db.url=jdbc:mysql://prod-db:3306/order-db

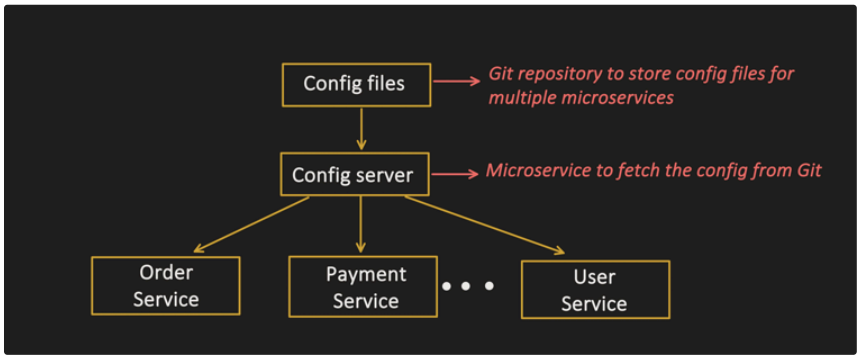
But this new DB is misconfigured and bcoz of that all the services crashes in production.

Now to fix:

- We must manually edit each affected service's config file.
- Rebuild the jar, redeploy the service.
- If multiple services were updated, you need to fix all.

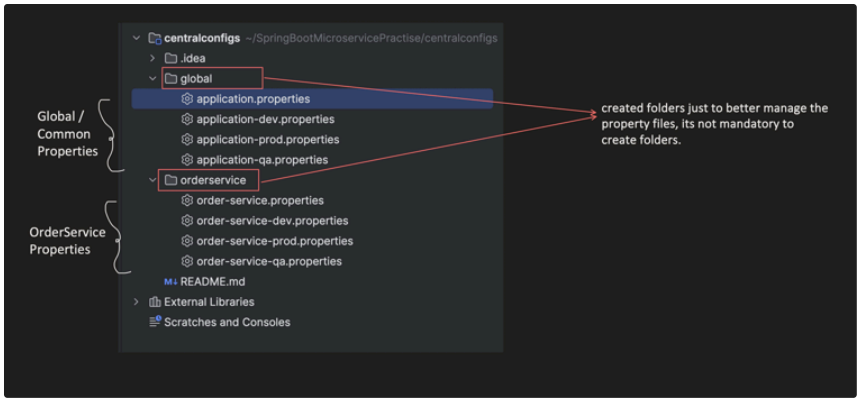
Solution: Centralized Configuration with Spring Cloud Config

Concept && Coding



Setup Step1:

Git repository, which just contain files:



Global / Common Properties:

- **application.properties** *(applied to all services and all profiles like dev, qa, prod etc.)*
- **application-{profile}.properties** *(applied to all services with specific profile either dev or qa or prod etc.)*

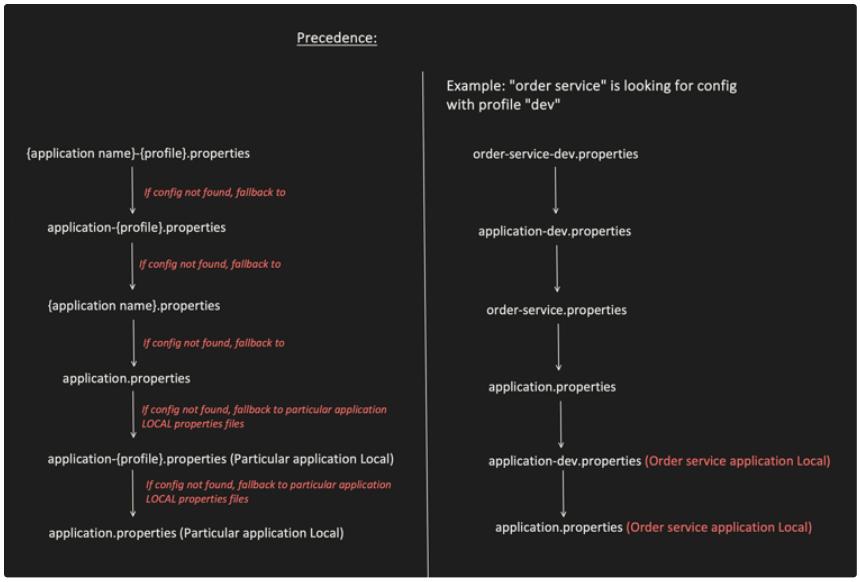
Service-Specific Properties:

- **{application name}.properties** *(applied to particular services and all profiles like dev, qa, prod etc.)*
- **{application name}-{profile}.properties** *(applied to particular services with specific profile either dev or qa or prod etc.)*

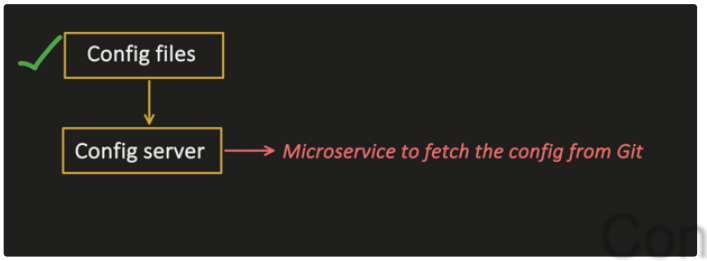
Concept && Coding

Already explained what is ***profile*** in depth, kindly check it out, if there is any doubt with this term:

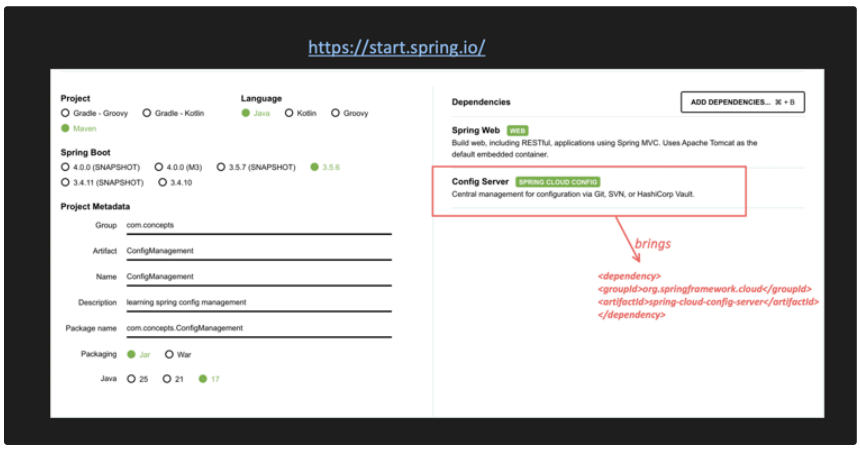
<u>Precedence:</u>



Setup Step2:



Concept && Coding



application.properties

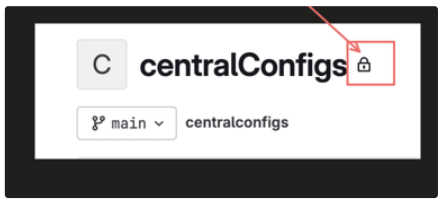
```
server.port=8888
<!--
  Public Git repo link of the centralized config
-->

spring.cloud.config.server.git.uri=https://gitlab.com/shrayansh8/centralconfigs
<!--
```

```
  Only required, when we have created folders in our central config git repo.
  Tells which all folders need to be scanned.
-->
  spring.cloud.config.server.git.search-paths=global, orderservice
<!--
  Download the properties from central config at the time of application startup
  , if its false, it will be downloaded at the time of first-request.
-->
  spring.cloud.config.server.git.clone-on-start=true
<!--
  This tell, which branch to use to fetch the properties.
  By default its "main" branch only.
-->
spring.cloud.config.server.git.default-label=main

```

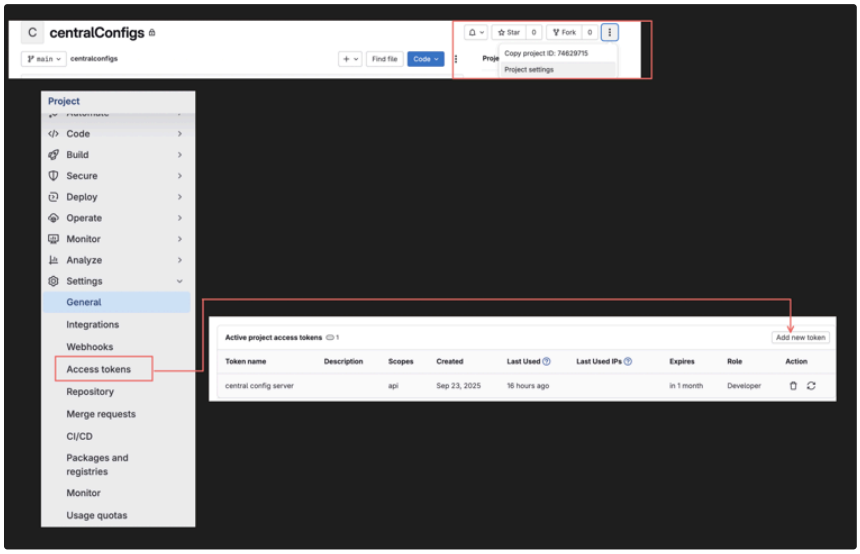
If Centralized Config Git Repo is private, then we have to provide username and password too:



Concept && Coding

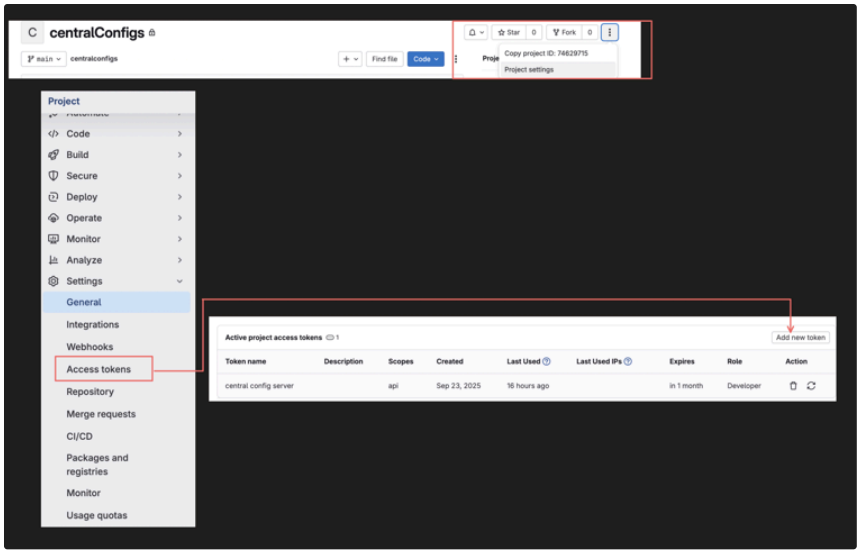
Gitlab:

Go to Project setting → Access Token → Add new token



GitHub:

Go to profile setting → Developer Setting → Personal Access Token → Fine Grained tokens

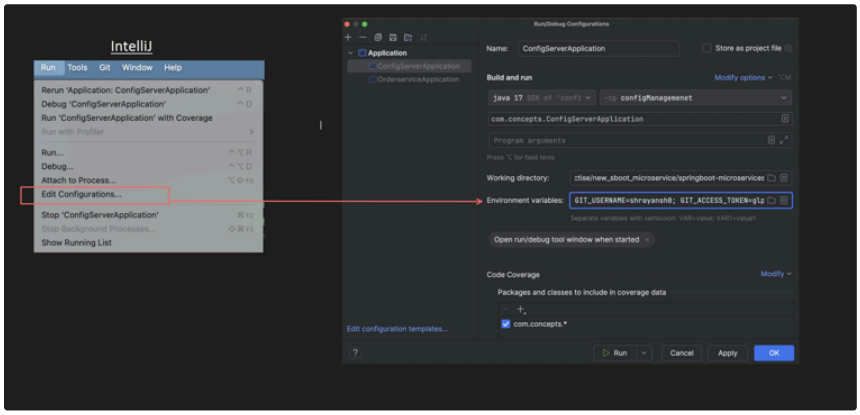



```
server.port=8888
<!--
Generally this is the username, in both GitLab and GitHub
-->

spring.cloud.config.server.git.uri=https://gitlab.com/shrayansh8/centralconfigs
<!--
  For safety purpose, we always set the tokens either in environment variables
      or key management server
-->
spring.cloud.config.server.git.username=${GIT_USERNAME}
spring.cloud.config.server.git.password=${GIT_ACCESS_TOKEN}
spring.cloud.config.server.git.search-paths=global, orderservice
spring.cloud.config.server.git.clone-on-start=true
spring.cloud.config.server.git.default-label=main

```

Concept && Coding



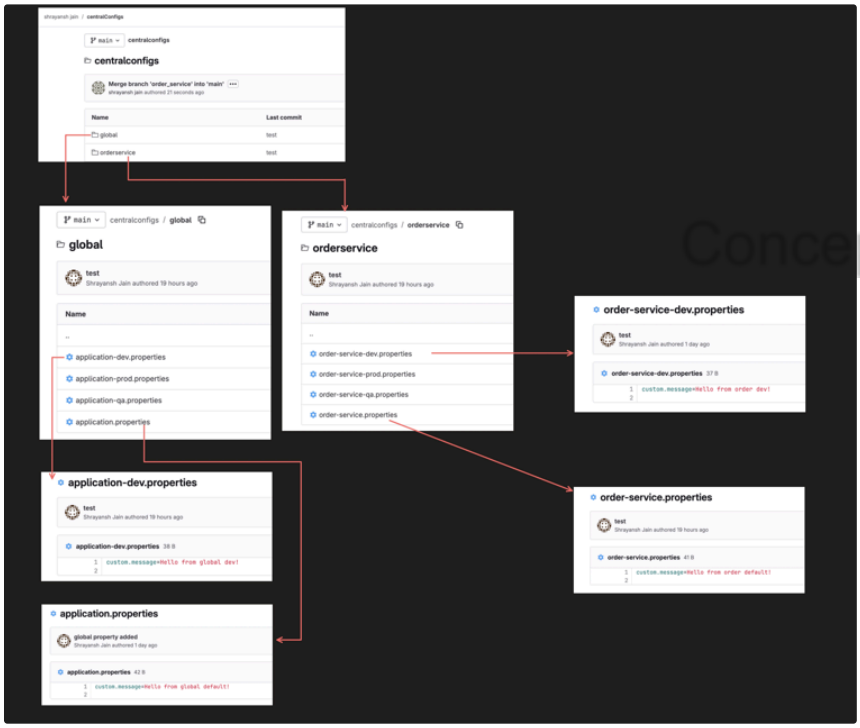
Turn this Springboot application into Central configuration server:

```
@SpringBootApplication
  /*
    Turn this Spring Boot app into a central configuration server.
```

```
4
5   Load properties file from central config repo during application
 startup
6     and expose config endpoints:
7
8     Expose endpoints like:
9
10    - {application}/{profile}/{label}
11  */
12 @EnableConfigServer
13 public class ConfigServerApplication {
14
15     public static void main(String[] args) {
16         SpringApplication.run(ConfigServerApplication.class, args);
17     }
18 }
19
```

Lets test till this point:

Updated the central config repo with properties file:



Start the Config Server:

```
1 @SpringBootApplication
2 @EnableConfigServer
3 public class ConfigServerApplication {
4
5     public static void main(String[] args) {
6         SpringApplication.run(ConfigServerApplication.class, args);
7     }
8 }
9
```

```
ConfigServerApplication   OrderserviceApplication

2025-09-24T12:43:54.743+05:30  INFO 31429 --- [          main] com.concepts.ConfigServerApplication     : No active profile set, falling back to 1 default profil
2025-09-24T12:43:55.023+05:30  INFO 31429 --- [          main] o.s.cloud.context.scope.GenericScope     : BeanFactory id=351a9386-b2a6-37d7-a466-a07e12d293b3
2025-09-24T12:43:55.113+05:30  INFO 31429 --- [          main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port 8888 (http)
2025-09-24T12:43:55.118+05:30  INFO 31429 --- [          main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2025-09-24T12:43:55.118+05:30  INFO 31429 --- [          main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/10.1.46]
2025-09-24T12:43:55.137+05:30  INFO 31429 --- [          main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2025-09-24T12:43:55.137+05:30  INFO 31429 --- [          main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in
2025-09-24T12:43:56.640+05:30  INFO 31429 --- [          main] o.eclipse.jgit.transport.TransportHttp   : URI 'https://gitlab.com/shrayanshB/centralconfigs': fol
2025-09-24T12:43:56.641+05:30  WARN 31429 --- [          main] pClientConfigurableHttpConnectionFactory : No custom http config found for URL: https://gitlab.com.
2025-09-24T12:43:57.320+05:30  WARN 31429 --- [          main] pClientConfigurableHttpConnectionFactory : No custom http config found for URL: https://gitlab.com.
2025-09-24T12:43:57.954+05:30  WARN 31429 --- [          main] pClientConfigurableHttpConnectionFactory : No custom http config found for URL: https://gitlab.com.
2025-09-24T12:43:58.458+05:30  WARN 31429 --- [          main] pClientConfigurableHttpConnectionFactory : No custom http config found for URL: https://gitlab.com.
2025-09-24T12:43:59.099+05:30  INFO 31429 --- [          main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8888 (http) with context path '/
2025-09-24T12:43:59.108+05:30  INFO 31429 --- [          main] com.concepts.ConfigServerApplication     : Started ConfigServerApplication in 4.56 seconds (proces
```

Hitting an endpoint: {application}/{profile}/{label}

localhost:8888/order-service/dev/main

Pretty print

```
{
  "name": "order-service",
  "profiles": [
    "dev"
  ],
  "label": "main",
  "version": "b369ddae81250530 2ae6b79f10ef2e56ebf5ee67",
  "state": "",
  "propertySources": [
    {
      "name": "https://gitlab.com/shrayanshB/centralconfigs/orderservice/order-service-dev.properties",
      "source": {
        "custom.message": "Hello from order dev!"
      }
    },
    {
      "name": "https://gitlab.com/shrayanshB/centralconfigs/global/application-dev.properties",
      "source": {
        "custom.message": "Hello from global dev!"
      }
    },
    {
      "name": "https://gitlab.com/shrayanshB/centralconfigs/orderservice/order-service.properties",
      "source": {
        "custom.message": "Hello from order default!"
      }
    },
    {
      "name": "https://gitlab.com/shrayanshB/centralconfigs/global/application.properties",
      "source": {
        "custom.message": "Hello from global default!"
      }
    }
  ]
}
```

Array of properties, in precedence order

1st precedence: {application name}-{profile}.properties

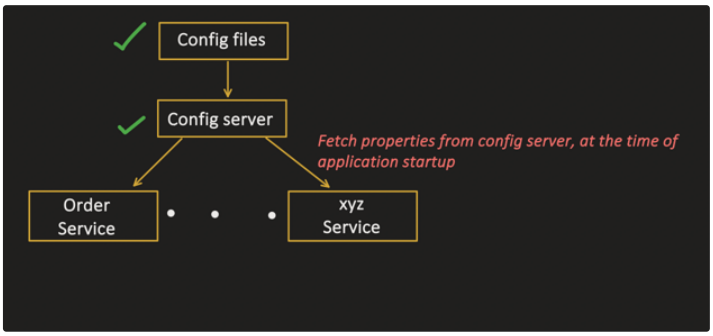
2nd precedence: application-{profile}.properties

3rd precedence: {application name}.properties

4th precedence: application.properties

Setup Step3: adding microservices

Concept && Coding



https://start.spring.io/

Project: Gradle - Groovy, Gradle - Kotlin, Maven

Language: Java, Kotlin, Groovy

Spring Boot: 4.0.0 (SNAPSHOT), 4.0.0 (M3), 3.5.7 (SNAPSHOT), 3.5.0, 3.4.11 (SNAPSHOT), 3.4.10

Project Metadata

Group	com.concept
Artifact	OrderService
Name	OrderService
Description	learning central config management
Package name	com.concept.OrderService
Packaging	Jar, War
Java	25, 21, 17

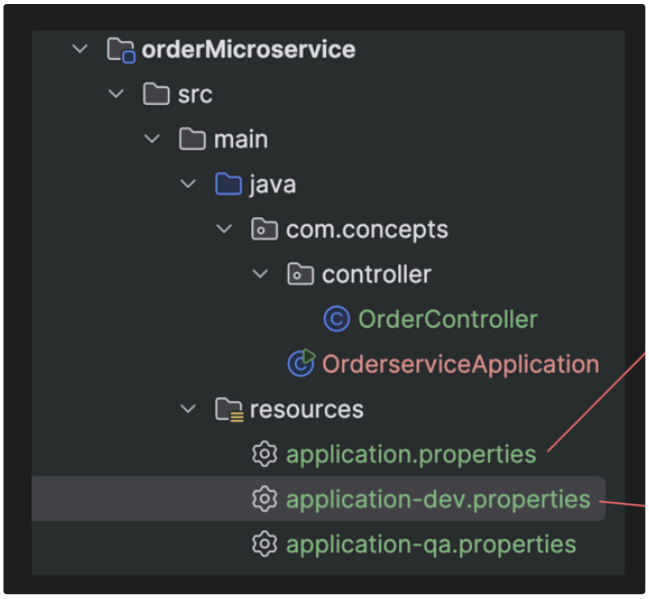
Dependencies — ADD DEPENDENCIES... ⌘ + B

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Config Client Actively Cloud Config
Client that connects to a Spring Cloud Config Server to fetch the application's configuration.

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

- *On startup, the app will contact a Config Server (enabled with @EnableConfigServer) and fetch its configuration.*
- *It automatically loads those properties into the app's Environment (as if they were local application.properties).*




```
spring.application.name=order-service
<!--
  This optional:configserver tells that, getting the properties from
  config server is NON FATAL, means even config server is down or
unreachable,
  we can stat the app using local properties.
-->
spring.config.import=optional:configserver:http://localhost:8888
spring.profiles.active=dev

#config for all profile (dev, qa, prod etc)
custom.message=Hello from local default!

```

Concept && Coding

```
custom.message=Hello from local dev!
```

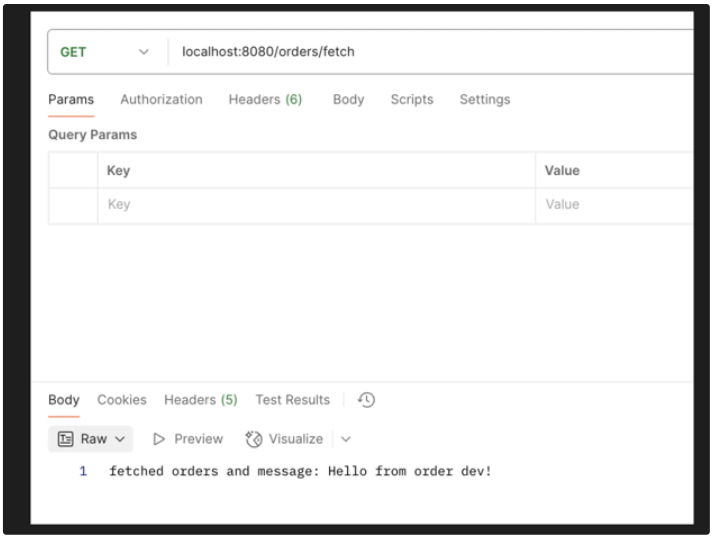
```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Value("${custom.message}")
    private String message;

    @GetMapping("/fetch")
    public String getOrders() {
        return "fetched orders and message: " + message;
    }
}

```

Start the Order application server and hit the controller api:



Now, one question might have come to mind that:

Say, our Order application is running and someone has updated the Central config for order service, do I need to restart the app every time?

Answer is **: NO**

Concept && Coding

There is a concept of @RefreshScope, which can load the properties dynamically (without restarting the order service)

But to understand **@RefreshScope** first, we need to understand **Actuator** first